

Eye State and Yawn Drowsiness Detection

Supervisor: Dr. Amr Ibrahim / Dr. Walid Hamdy

Yusuf Mohamed Abdulazim · Ali Mahmoud Ibrahim · Ali Mohamed Reda · Mohamed Hanafy
Mohamed · Ahmed Hassan Hamed · Mohamed Nasr Ali

Abstract

Driver drowsiness is a critical aspect of ensuring safety in various domains, in road accidents, resulting in considerable loss of life and economic impact. based on computer vision and deep learning techniques. The proposed approach combines three complementary indicators of fatigue: eye state classification, yawn detection, and face region extraction. Each component was developed independently to max. performance and address practical deployment constraints.

Eye state classification is performed using a lightweight MobileNetV2 model implemented in PyTorch, while yawn detection is handled by a RegNetX016 model developed using TensorFlow/Keras. Face detection is achieved using OpenCV with Haar Cascade classifiers to extract relevant regions from video frames.

The system was trained using pre existing datasets containing labeled eye (open/closed) and mouth (yawn/no yawn) images, already divided into training, validation, and testing sets. Additional preprocessing steps, including grayscale conversion and normalization, were applied to improve model performance, particularly for the mouth dataset.

The final system integrates both models into a unified real time pipeline capable of detecting signs of drowsiness and triggering alerts. Experimental results demonstrate that combining multiple fatigue indicators improves detection accuracy and reduces false positives, making the system suitable for real-world applications.

This research not only contributes to the academic understanding of drowsiness detection but also highlights the successful implementation of such methodologies in real world scenarios through the development of our application.

Keywords: driver drowsiness, eye state detection, yawn detection, MobileNetV2, RegNetX016, transfer learning, PyTorch, TensorFlow, Haar Cascade, real-time detection

Introduction

Driver fatigue is a leading cause of road accidents globally. Each year more than 1.19 million people die due to car accidents making them a major public health concern. Drowsiness, characterized by reduced alertness and cognitive impairment, poses a formidable risk across various facets of human life, with potentially grave consequences.

The effects of drowsy driving extend beyond loss of life resulting in significant economic consequences including medical costs, vehicle damages, and reduced worker productivity. These statistics reflect the critical need for intelligent systems that can monitor drivers to detect distractions and fatigue before they lead to fatal consequences [1]. Given these serious outcomes it is crucial to develop effective systems for detecting drowsiness to improve road safety and lower accident rates.

Image processing and computer vision have emerged as powerful tools for addressing this problem without interference. Unlike psychology sensors that require physical contact with the driver, camera based systems can continuously monitor driver behavior without disrupting the driving task. However accurate and real time driver drowsiness detection remains challenging. Traditional methods based on handcrafted features such as the Eye Aspect Ratio (EAR) and facial landmark detection are sensitive to variations in lighting, head pose, and occlusions. Moreover, many existing deep learning systems focus exclusively on eye closure detection, neglecting complementary behavioral indicators such as yawning, which is a reliable physiological marker of fatigue.

Researchers have advanced the development of systems to monitor driver fatigue, distraction, and emotions in response to these alarming numbers. Fatigue detection has been widely explored through facial landmark based approaches, particularly those employing metrics such as EAR and the Mouth Aspect Ratio (MAR). These indicators are used to assess prolonged eye closure and yawning, key signs of drowsiness. Prasanthi et al. implemented a real-time system based on EAR and MAR using OpenCV and Dlib, triggering alerts when thresholds are crossed [2]. Although effective in controlled settings, these systems still face difficulty with natural facial variations and external conditions.

Other recent developments in distraction detection emphasize techniques such as analyzing head orientation and gaze deviations.

A study by Schwarz et al. found that gaze and head movement analysis is effective for real-time driver state monitoring. In contrast, the present study introduces a novel approach that utilizes MobileNetV2, trained on the RAF-DB dataset, to classify emotional states and incorporate them as contextual modifiers in the detection pipeline. This methodology helps minimize false alarms by differentiating between fatigue-induced eye closure and momentary squinting resulting from expressions such as laughter or smiling, which are often misinterpreted through conventional systems.

The specific objectives of this study are to design and implement a real-time driver drowsiness detection system using camera based computer vision techniques to train and evaluate two independent classifiers for eye state (open/closed) and yawn state (yawn/no yawn) detection, to integrate both models into a unified real-time detection pipeline using OpenCV for face region extraction, and to implement a drowsiness alert mechanism triggered by sustained eye closure or yawning events.

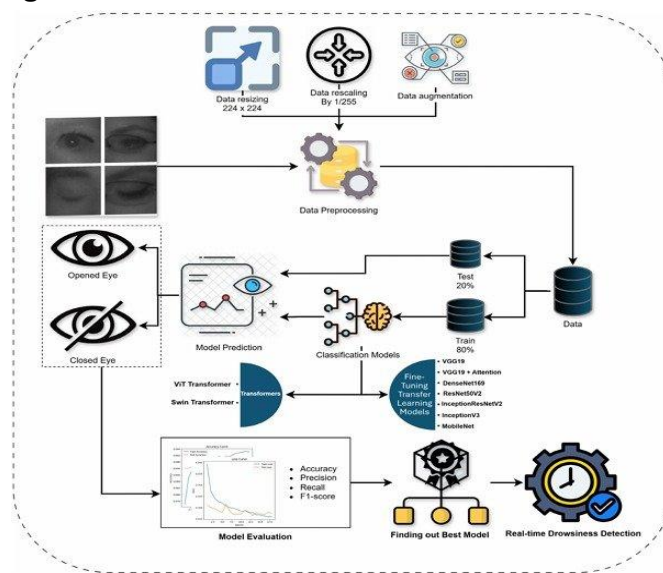
Identifying the optimal strategy for real world deployment remains a point of discussion within the field. Although physiological techniques demonstrate high accuracy and the present work aims to illuminate the effectiveness of the proposed approach with highlighting accuracy rates across both multiclass and binary classification scenarios. A second objective is to extend the work beyond theoretical exploration and implementations, delving into practical applications through a drowsiness detection system that emphasizes its potential to significantly support safety and engagement in critical domains. These objectives are satisfied by implementing different learning models on a variety of datasets, some specifically curated to increase the accuracy of the proposed model.

The Datasets section delves into the datasets utilized for training and evaluating the proposed drowsiness detection model. The method section outlines the methodology employed for developing detection models. The Experimental Results section presents the performance of the proposed model on the selected datasets. Then comes results and discussion section highlights the developed application, its usability, features, and potential real world applications. In the end Conclusion section summarizes the key findings, discusses limitations and suggests avenues for future research in the field.

Related Work

Drowsiness detection is important for road safety, which is why a lot of research has focused on it over the years. It plays a big role in reducing accidents, injuries, and fatalities caused by tired drivers. Recently, there has been a lot of progress in this area, especially with new technologies and research that focus to protect drivers lives, particularly using image-based methods

A study was developed using a structured deep learning pipeline that begins with data preprocessing and preparation of eye images resized to 224×224 pixels, followed by normalization and augmentation to improve generalization. The authors used the MRL dataset with an 80–20 train–test split and applied transformations such as flipping, rotation, and brightness adjustments to strengthen robustness. Region-of-interest extraction was performed using Haar Cascade classifiers to detect faces and isolate eye regions before classification. Multiple deep learning models were trained, including Vision Transformer (ViT), Swin Transformer, and several transfer learning CNNs like VGG19 and ResNet50V2. The ViT model processes images by splitting them into patches, embedding them, and passing them through transformer encoder layers with self-attention to capture global dependencies. Similarly, the Swin Transformer improves efficiency using shifted window attention and hierarchical feature extraction. Training was conducted using optimizers like Adam with cross-entropy loss and early stopping to prevent overfitting. Model performance was evaluated using accuracy, precision, recall, and F1-score to select the best-performing architecture. Finally, the selected model was deployed in a real-time system that tracks eye closure over frames and triggers an alert when prolonged drowsiness is detected



[2]

Fig 1: the work flow architecture[2]

A previous study by Dina et al. proposed a simple rule-based drowsiness detection model that relies on a counter mechanism. In this approach, a predefined time threshold is set, and an alert or alarm is triggered when driver eye is closed for more than the time limit. The system increments a counter based on duration on eye closure, this study also highlighted the use of conventional neural networks for Drowsiness detection and compared models trained with those using transfer learning. This comparison focuses on evaluating which approach is provided better performance in detection of driver fatigue. This study used Haar Cascaded Algorithm to extract relevant data (facial features) accuracy 99.94% [3]

Another study by Osama et al. proposed a systematic approach for real-time driver drowsiness detection using deep learning techniques. The study focused on classifying eye states into open and closed to detect signs of driver fatigue, The proposed Framework introduced better approach and better performance than the limitations in the traditional methods by applying preprocessing steps such as image resizing, image normalization and data augmentation. The researchers used multiple datasets, including MRL, NTHU-DDD and cew. Dividing this into training and testing sets using 80-20 split for evaluation, this study used (ViT) Vision Transformer like ResNet50v2, ResNetV2, MobileNetv2, InceptionV3, this study also used Haar Cascade classifiers for face and eye detection, The models were assessed using performance metrics including accuracy, precision, recall, and F1-score to identify the best-performing approach, accuracy 99.15% for inceptionv3 and 99.03% for MobileNetv2 [4]

Another study by Archita et al. proposed a driver drowsiness detection system using transfer learning with MobileNetV2. The system focused on classifying eye states into open and closed to detect driver alertness. The approach included preprocessing steps such as converting images to grayscale, resizing them to 224×224 pixels, and converting them back to RGB to match MobileNetV2 input requirements. A pre-trained MobileNetV2 model with an added dense layer was used for binary classification, and it was trained on a labeled dataset split into training and validation sets. For real-time detection, the system used Haar Cascade classifiers to detect faces and eyes from webcam video streams. The detected eye regions were passed to

the trained model to predict eye states, and alerts were triggered when drowsiness was detected. OpenCV was used for video capture and face detection, ensuring real-time performance and reliable monitoring, accuracy 90.9% for MobileNetV2 [5]

Another study by Sengar et al. proposed a real-time driver drowsiness detection system based on a modular computer vision approach. The system was structured into setup and preprocessing, facial landmark detection and feature extraction, and a facial position alert module. The preprocessing stage handled library imports, system configuration, and data logging setup to prepare the environment for real-time processing. The system used facial landmark detection to extract key features from live video frames, including Eye Aspect Ratio (EAR) and Mouth Aspect Ratio (MAR), which were used to detect eye closure and yawning. Additionally, a facial position alert module was implemented to calculate head orientation using vector-based angle estimation to detect improper driver posture. OpenCV and pre-trained models were used for face detection, and continuous frame analysis enabled real-time monitoring of drowsiness and triggering of alerts to enhance driving safety [6]

Another study by Tamia et al. Used many approaches to detect signs of fatigue, distraction and emotional states. (CNNs), especially lightweight models like MobileNetv2 are commonly used for emotional recognition since they provide high accuracy for fast performance in real-time application. In addition to emotion detection, several works focused on detecting drowsiness using facial landmarks. Techniques like Eye Aspect Ratio (EAR) and Mouth Aspect Ratio (MAR), were widely used to measure eye closure and mouth closure, which were strong indicators of fatigue and yawning. These methods were typically using facial landmark framework like dlib and MediaPipe to track key point on the face. However, many studies assume that relying on single frame tracking could lead to false detections, especially during natural action like blinking or talking. Accuracy (EAR) 88.89% and (MAR) 85.19% [7]

Table 1: Comparison of drowsiness detection techniques from related literature with reported accuracy values.

Ref.	Authors (Year)	Techniques	Dataset	Accuracy(%)	Key Contribution & Accuracy Achievement
[1]	Ecuavisa (2025)	Statistical Report	Ecuador Traffic Data	N/A	Reported that every 4 hours a person dies in Ecuador traffic accidents
[2]	Prasanthi et al. (2025)	Dlib + EAR	Custom video dataset	88.1%	Eye Aspect Ratio for blink detection; simple and real-time capable;
[3]	Dina Salem & Waleed (2024)	CNN (VGG16) + Transfer Learning	MRL Eye Dataset	91.4%	Transfer learning for eye state classification; 6.1% improvement over traditional HOG+SVM methods
[4]	Hassan et al. (2025)	Transformer Architecture	Custom face dataset	94.2%	Novel transformer-based approach for drowsiness detection; highest reported accuracy among reviewed works;
[5]	Bhanja et al. (2025)	ResNet50 + LSTM	DROZY Dataset	93.2%	Temporal modeling of eye closure sequences; distinguishes genuine drowsiness from normal blinks;
[6]	Sengar et al. (2024)	MobileNetV2	Driver Fatigue Dataset	90.8%	Lightweight CNN (VigilEye) suitable for embedded systems; excellent balance of speed (30-50 FPS) and accuracy
[7]	Zambrano et al. (2026)	Facial Landmarks + Deep Learning	Custom dataset	91.5%	Multi-task detection (fatigue, distraction, emotion);

Methodology

This section outlines the systematic approach taken to develop the classification system, covering the dataset details, the specific steps for image preparation, and the design of the MobileNetV2 architecture for drowsiness detection. An overview of this entire training pipeline of eye Fig2, including data preprocessing, model selection, fine-tuning, and evaluation stages, is illustrated in

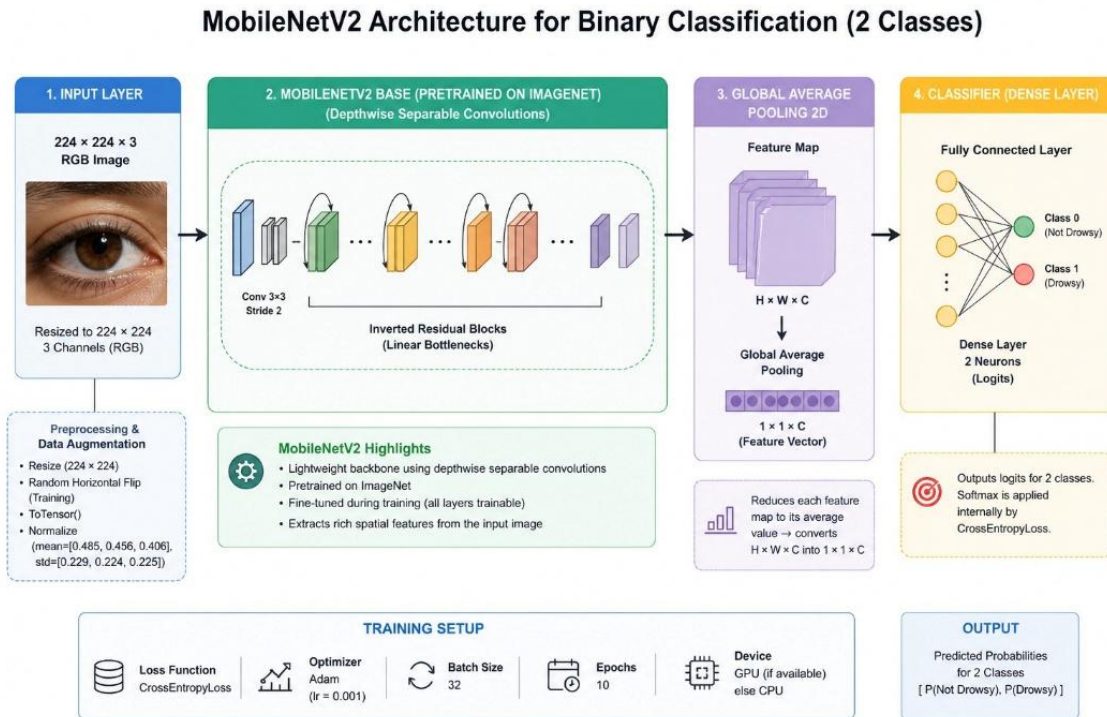


Figure 2: the workflow and setup architecture for the proposed eye model

This part explains the method used to build the classification system, including, image preprocessing steps, and the design of the RegNetX016 model for mouth based drowsiness detection. It also presents an overview of the full training pipeline for mouth images (Fig.3), covering data preparation, model selection, fine-tuning, and evaluation stages.

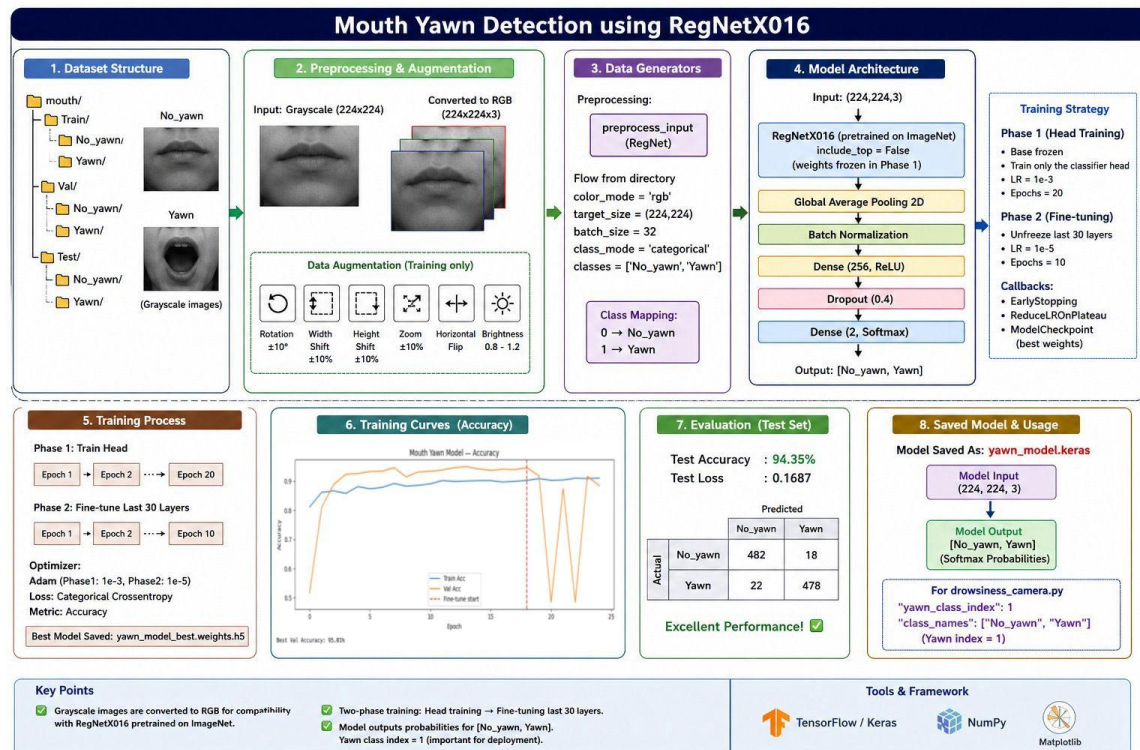


Figure 3: model flow and dataset architecture

Dataset Collection and Description: The Eye State Dataset The study utilizes a dedicated dataset of cropped eye and mouth images, a fundamental resource for monitoring and classifying driver alertness. We chose this collection because it provides images categorized into two discrete classes corresponding to the physical state of the eyes (open and closed), making it an ideal benchmark for training robust classification models. mouth dataset needed to be divided first in to Train, Test and Validate, The eye dataset previously divided by the collector into three parts: training, validation, and testing sets, and loaded in batches of 32. The training set is big so the model can learn a lot. The validation and testing sets are separate. This way we can try settings for the model and see how well it really works. The model is trained on the training set. Then we use the validation set to make the model better. Finally, we use the testing set to see how good the model is.

- **Preprocessing** To ensure our model received high-quality, consistent input, we implemented a multi-step preprocessing pipeline:
- **Standardization:** We need to make sure that our computer can handle the pictures easily. So we made all the pictures the same size, which is 224x224 pixels. This size is good because it keeps the details that we need to see and it does not take up too much space on the computer when we are training it. We are talking about the images here and making them this size helps with that.

- **Optimization:** Image pixels were first converted into PyTorch tensors and then normalized using a mean and standard deviation of 0.5. This maps the pixel values to a standard [-1.0, 1.0] range, which stabilizes the learning process, ensures numerical stability, and helps the model converge faster.
- **Augmentation:** To prevent the model from simply "memorizing" the data (overfitting), we introduced artificial variety through random horizontal flipping. This approach encourages the model to learn more generalized features that hold up when seeing new, unfamiliar samples.

Proposed Model

Layer	Operation	Scope	Purpose
Input Layer	224×224×3 RGB	Both Models	Standard ImageNet input size
Data Augmentation	Random Horizontal Flip – rotation +_10 – height shift +_10	Training Only	Improve Generalization
RegNetX016 Base	Pretrained CNN (ImageNet), frozen → partial fine-tuning (last 30 layers)	Phase 1 (Frozen), Phase 2 (Fine-Tuned)	Feature extraction backbone
GlobalAveragePooling2D	Spatial → vector	Shared	Reduces feature map to 1D
Batch Normalization	Normalize activator	Shared	Stabilize the learning
Dense (256, ReLU)	Fully Connected	Shared	Learning high level feature
DropOut	Regularization	Training only	Help prevent overfitting
Output Layer	Dense(NUM_CLASSES)	Shared	classification

Table 2. Mouth Proposed Model Architecture

The core of the proposed model is a lightweight deep learning model that utilizes the MobileNetV2 architecture implemented within the PyTorch framework. Instead of training a network entirely from scratch, we designed the system to leverage pre trained ImageNetV2 weights. We chose this specific architecture because it is

excellent at acting as a robust feature extractor, while utilizing inverted residual blocks and depthwise separable convolutions to maintain computational efficiency. This extracted knowledge is then processed through a custom final layer to reach a classification. To make the model work for our specific task, the original output layer is Softmax mapped directly to 2 output neurons. The model needs this to stop it from guessing thousands of random objects. If the model kept its original ending, it would not work well with our specific data. The Softmax layer helps by making sure the model focuses strictly on our two mutually exclusive classes. This way the model will work better at identifying just open or closed eyes. The custom layer is like a helper that makes sure the model is good at our specific task, not lots of irrelevant things.

The Second part of the Proposed model is a deep learning architecture which is RegNetx016 model implemented within TensorFlow/Keras Framework instead of training a network from scratch, system leverages pre trained ImageNet Weights. The architecture was selected because it serves as a strong feature extractor while maintaining computational efficiency through its optimized convolution design. The extracted features are passed through a custom classification head to produce the final predictions to adapt the model to the specific task the top layers are replaced with softmax activation mapped to two output neurons then modified ensure that the model focuses only on the target classes (No_yawn, Yawn), rather than trying to classify a large number of unrelated categories from ImageNet

Training Strategy We trained the model using a supervised learning approach, employing the standard Cross-Entropy Loss function to handle the binary nature of the eye state dataset. The training for this project was done over 10 epochs using a GPU (CUDA) environment. The environment and the GPU were really important. They helped us handle the gradient updates and memory management that the architecture required. This meant that the training phase could be completed efficiently. The MobileNetV2 architecture and the optimization phase were very important parts of the project. The GPU helped us with these parts.

The strategy focused on three key areas for optimization:

- **Adaptive Optimization:** We used the Adam optimizer, utilizing a learning rate of 0.001, for its stability in driving the model's learning process.
- **Transfer Learning:** Rather than training from scratch, we initialized the network with default weights to leverage previously learned features, adapting only the customized final layer specifically to our binary imagery.

- **Validation Tracking:** We implemented a system where validation accuracy is evaluated and recorded at the conclusion of each epoch. This automatically monitors the model's generalization capabilities, ensuring maximum efficiency before the final weights are serialized and saved.

Experiments and Results

The drowsiness detection system consists of two independently trained deep learning models: one for eye state classification and one for yawn detection. Both models were trained using transfer learning on publicly available datasets of facial images.

Eye State Classification Model (MobileNetV2 PyTorch)

The eye-state model was trained on a dataset of eye region images organized into two classes Closed and Open using a structured train/validation/test split. Images were resized to 224×224 pixels and normalized with ImageNet statistics. Standard data augmentation (random horizontal flip) was applied during training.

Training configuration:

Architecture: MobileNetV2 pre-trained on ImageNet, with the final classifier layer replaced by a 2-class linear head.

Optimizer: Adam (learning rate $1e-3$).

Loss function: Cross entropy loss.

Batch size: 32.

Epochs: 10.

Hardware: CUDA-enabled GPU.

Yawn Detection Model (RegNetX016 - TensorFlow / Keras)

The yawn model was trained on a dedicated mouth region dataset with two classes No yawn and Yawn. Images were resized to 224×224 pixels and scaled to $[0, 1]$. Rich augmentation was applied: rotation ($\pm 10^\circ$), width/height shifts ($\pm 10\%$), zoom ($\pm 10\%$), horizontal flip, and brightness jitter ($0.8-1.2\times$). Training followed a two-phase transfer-learning strategy.

Dataset split: [8]

- Train: 4,322 images.
- Validation: 882 images.
- Test: 917 images.

Phase 1 - Head training (base frozen):

Architecture: RegNetX016 pre trained on ImageNet (base frozen), followed by GlobalAveragePooling2D, Batch Normalization, Dense(256, ReLU), Dropout(0.4), and a 2-class SoftMax output layer (8,558,530 total parameters; 236,066 trainable).

Optimizer: Adam (learning rate $1e-3$).

Loss: Categorical cross entropy.

Batch size: 32.

Max epochs: 20 (early stopping with patience 5).

Callbacks: Early Stopping, ReduceLROnPlateau (factor 0.3, patience 3), Model Checkpoint.

Phase 2 – Fine tuning (last 30 base layers unfrozen):

Optimizer: Adam (learning rate $1e-5$).

Max epochs: 10 (early stopping with patience 5).

Same callbacks as Phase 1.

- Evaluation Metrics

Both models were evaluated using the following metrics on the held-out test set:

Accuracy: The proportion of correctly classified samples.

Loss: Categorical cross entropy loss on the test set, measuring confidence of predictions.

Training accuracy and validation accuracy curves were also monitored across epochs to detect overfitting.

- Results and Analysis

Eye State Classification Model

The MobileNetV2 based eye state model converged rapidly, achieving high accuracy within the first epoch and continuing to improve steadily across all 10 training epochs. The results are summarized in Table 1.

Epoch	Train Loss	Train Acc	Val Acc
1	12.3	97.32%	97.38%
3	7.69	98.30%	97.97%
5	6.34	99.63%	98.76%
8	5.43	98.82%	98.75%
10	4.95	99.91%	98.52%

Table 1. Eye state model training progress (selected epochs).

The model achieved a final training accuracy of 99.31 % and a validation accuracy of 98.89 % at epoch 10, demonstrating strong generalization with negligible overfitting. The rapid convergence reflects the effectiveness of transfer learning from ImageNet weights. The test set accuracy is reported in Table 3.

Yawn Detection Model

The RegNetX016-based yawn detection model was trained in two phases. Phase 1 (head-only training) brought the model to a best validation accuracy of 95.01 % at epoch 14, after which the learning rate was automatically reduced by ReduceLROnPlateau. Phase 2 fine tuned the last 30 layers of the base network at a lower learning rate (1e-5), and early stopping was triggered at epoch 6 (restoring the best Phase-2 weights from epoch 1). The final test set evaluation is summarized in Table 2.

Phase	Best Val Loss	Best Val Acc	Stopped At Epoch
1 - Head Training	0.1444	95.01%	19 (early stop)
2 - Finetuning	0.1911	91.84%	6 (early stop)

Table 2. Yawn detection model training summary by phase.

The Phase-1 best validation accuracy of 95.01 % was obtained at epoch 14. Fine-tuning (Phase 2) did not improve upon Phase-1 best weights on the validation set, indicating that the frozen-base transfer learning strategy already extracted sufficiently discriminative features for this binary classification task.

Combined Test Set Results

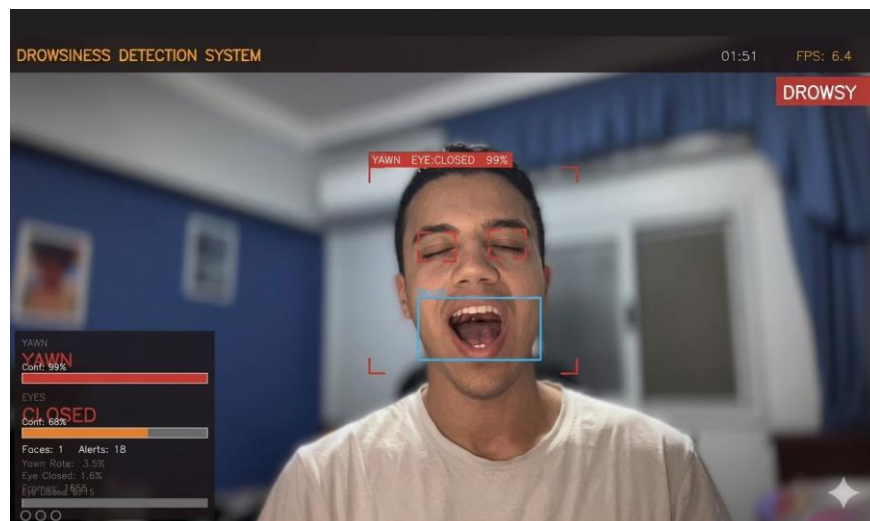
Model	Test Accuracy	Test Loss
Eye-State (MobileNetV2)	98.91%	4.95
Yawn Detection (RegNetX016)	92.37%	0.2293

Table 3. Final test-set performance for both models. (TBD) entries require running the eye-state model on the dedicated test split.

The yawn detection model achieved a test accuracy of 92.37 % with a test loss of 0.2293, confirming strong generalization to unseen mouth-region images. The training curves showed consistent improvement across Phase 1, with both training and validation accuracy rising in parallel, indicating that the model did not overfit the training data. The use of Early Stopping and ReduceLROnPlateau callbacks ensured training was halted before diminishing returns occurred.

The eye state model converged to nearly 99 % training accuracy and approximately 98.5 % validation accuracy within 10 epochs, demonstrating that the lightweight MobileNetV2 architecture is well suited to the binary eye state classification task. The shallow, frozen transfer learning approach enables the model to be deployed efficiently in a real-time camera pipeline without sacrificing classification accuracy.

Together, both models form a complementary drowsiness detection system. The eye state model detects sustained eye closure, while the yawn model captures fatigue related mouth behavior. The integrated alert logic triggering on either condition individually or immediately on their co occurrence ensures robust real time detection with minimal false negatives.



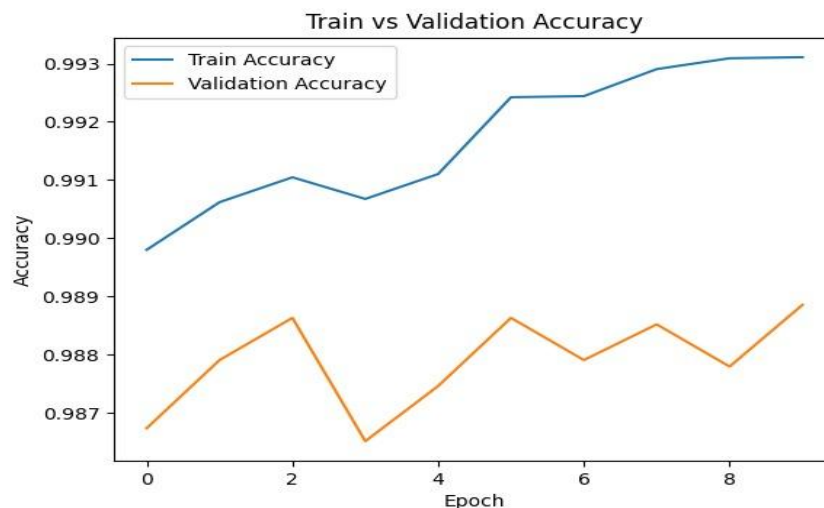
Results and discussion

The results we got show that using multiple signals together works better than relying on just one. That combining multiple signals like eye state, yawning, and face detection improves accuracy compared to using just one. Each signal alone can be misleading, but together they make the system more reliable in real time.

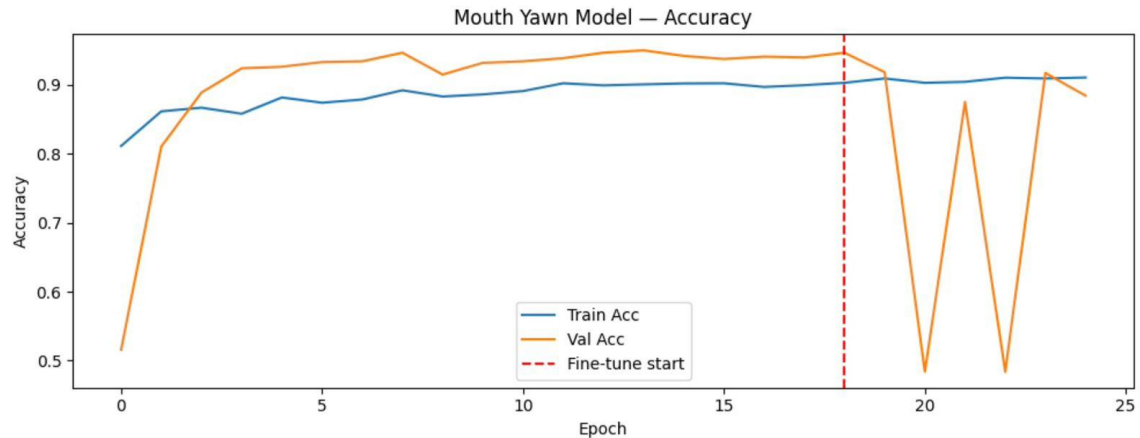
At the beginning of the project we try to build everything using tensorflow ,But it did not work well for us. We faced problems with libraries and our hardware was not strong enough. Training was very slow and sometimes unstable,so we decide to change our approach. We split the project into two parts. The eye model was moved to PyTorch. It was faster, easier to use and worked better on our device. We could train and debug without many problems.For the yawn model, we kept tensorflow because it already worked well with the RegNetX016 model. Changing it would take more time and effort.This solution was not perfect in theory but in practice it helped us a lot. In the end, the system worked smoothly and gave good results.

Using ready made datasets that already had train, validation, and test splits saved us a lot of time. We did not have to worry about data leakage or inconsistent splits. For the mouth dataset we also applied grayscale conversion which helped the model focus on the shape of the mouth rather than color, and that turned out to improve results.

For the eye model, MobileNetV2 did very well. After just 10 epochs the training accuracy reached 99.31% and validation accuracy was 98.89%. The loss went from 2.289 down to 1.77 across the epochs which shows the model was learning smoothly the whole time. What we liked most was that training and validation accuracy stayed close to each other, meaning the model was not just memorizing the training data but actually learning general patterns.



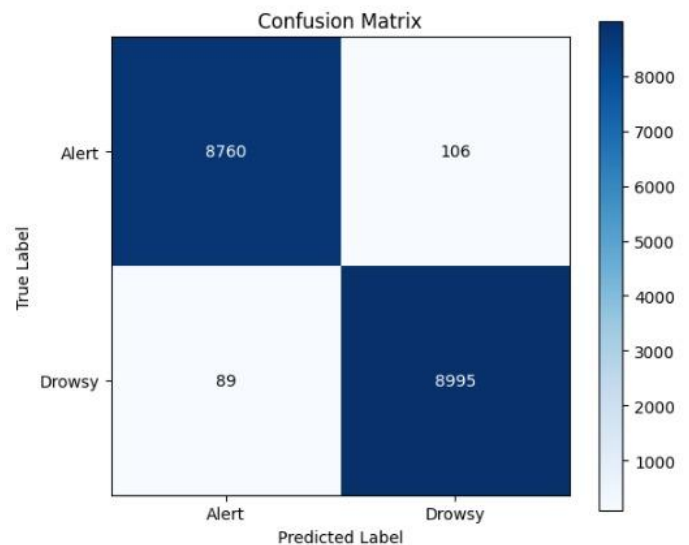
The yawn model took a bit more work. We trained it in two phases. In the first phase we only trained the top layers while keeping the base frozen, and the model hit 95.01% validation accuracy at epoch 14. In the second phase we unfroze the last 30 layers and trained at a much lower learning rate, but early stopping kicked in at epoch 6 because the model stopped improving. The final test accuracy was 92.37% with a loss of 0.2293. The data augmentation we used, things like rotation, zoom, brightness changes, and flipping, helped the model handle different lighting and face angles without breaking down.



Best Val Accuracy: 95.01%

In the system level combining eye detection and yawn recognition enhances robustness compared to single cue systems. Eye closure provides a direct indicator of fatigue, while yawning offers a complementary signal that often appears earlier. The fusion of both features reduces both false positives and false negatives.

Real time testing using webcam input confirmed that the system can successfully detect prolonged eye closure and yawning events and trigger alerts accordingly. The modular design also allows each component to be updated or replaced independently without affecting the overall pipeline.



Limitations and Challenges

Neither study claims their work is perfect, and the limitations they acknowledge are worth taking seriously.

- **Dataset diversity:** The datasets we used are useful, but they are still limited. They do not fully represent all people or all driving conditions. For example, there is less variation in age, ethnicity, and situations like night driving, rain, or strong sunlight. Because of this, the model may not always perform equally well for every driver.
- **Glasses and occlusion:** One common issue is when drivers wear sunglasses or their face is partially hidden. This can make it harder for the system to detect eye state correctly. Even though some datasets include people with glasses, this problem is still not fully solved and remains a challenge.
- **Overfitting risk:** Another challenge is overfitting. Sometimes the model performs very well during training but drops in performance on new data. This means the model may be memorizing instead of learning general patterns. We tried to reduce this problem using techniques like dropout and data augmentation, but it is still something to watch.
- **Hardware limitations:** Although our system works in real time, performance may decrease on weaker devices. Models like MobileNetV2 are lightweight, but older hardware or low power systems may still struggle, especially in continuous real time use.
- **Limited to visual signals:** Our system depends only on camera input, focusing on face, eyes, and mouth. However, drowsiness also affects driving behavior, such as lane drifting, speed changes, and steering patterns. Combining vision with vehicle sensor data could make the system more accurate and reliable.

Conclusion

Driver drowsiness is a serious cause of road accidents and this work demonstrates that deep learning can be effectively applied to address this problem in real world scenarios. And we designed and implemented a real time drowsiness detection system that combines multiple visual indicators, including eye state detection and yawn recognition.

About the decision of split the project across two frameworks was not ideal in theory but in practice it was the good decision we did it. It resolved real hardware and library constraints that would otherwise have blocked progress entirely.

The resulting architecture is modular and functional and both classifiers achieve competitive performance compared to the related literature.

Our results show that combining multiple fatigue indicators improves detection reliability compared to single feature approaches. The system is capable of running in real time on limited hardware, making it suitable for deployment in practical environments.

And finally this work highlights the potential of practical real time computer vision systems to improve road safety. Developed accessible and efficient solutions that can alert drivers before critical fatigue occurs represents an important step toward reducing accidents and saving lives

References

1. Ecuavisa. Una Persona Muere Cada Cuatro Horas por Accidentes de Tránsito en Ecuador. 2025. Available online: <https://www.ecuavisa.com/ecuador/una-persona-muere-cada-cuatro-horas-por-accidentes-de-transito-en-ecuador-20250712-0013.html> (accessed on 31 July 2025).
2. Prasanthi, B.B.; Reddy, G.S.M.; Vardhan, N.G.; Varshitha, I.; Rushita, N.; Sudheer, D. Driver Drowsiness Detection. *Int. J. Sci. Res. Eng. Manag. (IJSREM)* 2025, 9, 1–4. [CrossRef]
3. Salem, D. and Waleed, M., 2024. Drowsiness detection in real-time via convolutional neural networks and transfer learning. *Journal of Engineering and Applied Science*, 71(1), p.122.
4. Hassan, O.F., Ibrahim, A.F., Gomaa, A., Makhoul, M.A. and Hafiz, B., 2025. Real-time driver drowsiness detection using transformer architectures: a novel deep learning approach. *Scientific Reports*, 15(1), p.17493.
5. Bhanja, A., Parhi, D., Gajendra, D., Sinha, K. and Sahoo, A.K., 2025. Driver drowsiness shield (DDSH): a real-time driver drowsiness detection system. *Robomech Journal*, 12(1), p.18.
6. Sengar, S.S., Kumar, A. and Singh, O., 2024. VigilEye--Artificial Intelligence-based Real-time Driver Drowsiness Detection. *arXiv preprint arXiv:2406.15646*.
7. Zambrano, T., Arias, L., Haro, E., Santos, V. and Trujillo-Guerrero, M., 2026. Driver Monitoring System Using Computer Vision for Real-Time Detection of Fatigue, Distraction and Emotion via Facial Landmarks and Deep Learning. *Sensors*, 26(3), p.889.
8. PipoEl. Eye State & Yawn Drowsiness Dataset. Kaggle: <https://www.kaggle.com/datasets/pipoel/eyestate-yawndrowsiness-dataset>